Soup Kernel Challenge: Оптимизация Qwen2.5 LoRA на Kaggle T4

Decentrathon 6.0 | Слайд 1: Задача и профилирование узких мест

Аппаратный контекст

- **GPU:** Бесплатная Kaggle Tesla T4 (15 ГБ VRAM, архитектура Turing sm_75 без поддержки bfloat16).
- **Модель:** Qwen2.5-1.5B в 4-битном формате NF4 (библиотека bitsandbytes).
- **Задача:** Обучение LoRA (r=16 на q_proj, v_proj), батч 4, seq_len 512, 64 шага, fp16.
- **Проблема:** На T4 обучение упирается в потолок VRAM — невозможно увеличить батч или длину контекста.

Проблема базового Soup

- **Время обучения:** 84.6 с на стандартной сессии Kaggle.
- **Пиковая память:** 14 345 МиБ из 15 ГБ (~95% доступной VRAM) — критическая близость к OOM.
- **Запас оптимизации:** Unsloth на аналогичной LoRA занимает около 2.3 ГБ (в 6 раз меньше).
- **Цель хакатона:** Ускорить обучение и радикально снизить память собственными Triton-кернелами.

Профиль узких мест (Soup)

- **44% времени:** мелкие поэлементные операции и касты FP32↔FP16 из-за перевода весов в FP32 в PEFT на sm<80.
- **7% времени и ~1.2 ГБ:** полный тензор логитов по словарю 152k ради подсчёта энтропии.
- **Лишние активации:** MatMul4Bit и Qwen2MLP сохраняют входы и тензоры [B·S, 8960] на всех 28 слоях.
- **Холостой ход:** вычисление внимания на pad-токенах разной длины внутри батча.

Архитектура решения: 7 Triton-кернелов и обвязка

Decentrathon 6.0 | Слайд 2: Что перенесли в Triton и как устроен пайплайн

7 собственных @triton.jit кернелов (10.6% времени)

- **qwen2_online_ce_entropy_fwd/bwd_kernel (fused_ce.py):** онлайн log-sum-exp, loss по целевому токену, точная энтропия и accuracy тайлами по 2048. Полный тензор логитов 152k не материализуется! Градиент пишется in-place.
- **qwen2_nf4_dequant_kernel (nf4_mlp.py):** векторная распаковка 4-битных весов в fp16 с честным double quantization в переиспользуемые scratch-буферы.
- **qwen2_rmsnorm_fwd/bwd_kernel (rmsnorm.py):** дисперсия в fp32, вывод в fp16 за один проход по строке. В backward сохраняется только rstd.
- **qwen2_rope_fwd/bwd_kernel (rope.py):** страйдовый поворот Q и K голов одним запуском без промежуточных срезов и копий памяти.

Оптимизации autograd и пайплайна

- **Чанкование Cross-Entropy по 256 строк:** через ChunkedHeadLoss со статическим fp16-буфером 74 МБ вместо 1.2 ГБ. Градиент dX считается сразу в forward.
- **Within-batch Unpadding (varlen):** удаление pad-токенов перед attention со склейкой в $[1, \Sigma L_i]$. Токены и порядок не меняются (rewind.jsonl идентичен базе).
- **Компактный MLP:** сохранение $gu = x \cdot W_gate_up^T$ для всех 28 слоев устраняет повторный тяжелый GEMM в backward, а silu_backward считается in-place.
- **Единый FP16 residual-поток:** замороженные веса оставлены в fp16, обучаемые LoRA — в fp32. Устранены паразитные касты.

Результаты бенчмарка: замер в Kaggle-ноутбуке

Decentrathon 6.0 | Слайд 3: Сравнение метрик и проверки по регламенту

ВРЕМЯ SOUP TRAIN

84.6 с → 62.5 с

Ускорение 1.352x (на 35.2% быстрее)

ПИК ПАМЯТИ GPU

14 345 → 2 759 MiB

В 5.199 раза меньше памяти

КАЧЕСТВО (EVAL LOSS)

2.04968 → 2.04976

+0.004% дельта (лимит ≤ 0.5%)

ДОЛЯ TRITON-КЕРНЕЛОВ

10.6% времени

Порог ≥ 5.0% выполнен

Проверки регламента: check_rules.py (ALL CHECKS PASSED — 10 из 10)

✓ **1.** Базовый и оптимизированный прогоны завершены.

✓ **2.** Сторонние библиотеки отсутствуют: loaded: none, imported: none.

✓ **3.** Все 7 собственных @triton.jit кернелов работают в обучении.

✓ **4.** Доля кернелов: 10.6% от времени шагов 3+ (двукратный запас).

✓ **5.** Результаты, токены, шаги и rewind.jsonl строго идентичны.

✓ **6.** Кривая loss повторяет базу (первый шаг 0.03%, среднее 0.29%).

✓ **7.** Качество сохранено: eval loss +0.004% при допустимых +0.5%.

✓ **8.** Не хуже базы: time x0.740, memory x0.192.

✓ **9.** Чистый замер на свободной Tesla T4 с аллокатором harness.

✓ **10.** Регрессионные тесты Soup: 79 passed. Юнит-тесты: 5 passed.

Честный анализ ограничений и перспективы развития

Decentrathon 6.0 | Слайд 4: Архитектурные компромиссы и точки роста

Честные архитектурные компромиссы

- **Не писали собственный GEMM на Triton:** cuBLAS от NVIDIA на Tensor Cores Turing уже оптимизирован до предела. Писать тривиальный matmul на Triton ради формального времени не имело смысла.
- **Без системного Gradient Checkpointing:** Soup поддерживает готовый флаг gradient_checkpointing, но мы оптимизировали память без него — за счёт управления временем жизни буферов и сброса промежуточных активаций.
- **Шум облачной среды Kaggle:** Время базы между сессиями колеблется от 79 до 90 с из-за фоновой нагрузки облака. Однако относительное ускорение стабильно держится на уровне ~1.35x, а экономия памяти — строго 5.2x.

Дальнейшие шаги (Next Steps)

- **1. Fused Dequant-GEMM в регистрах Triton:** объединение распаковки 4-битных весов прямо в тайлы GEMM. Это полностью устранит запись fp16-весов в глобальную память VRAM.
- **2. Кастомный Varlen Attention кернел:** замена цикла по последовательностям поверх PyTorch SDPA на единый оптимизированный FlashAttention-кернел с поддержкой cu_seqlens.
- **3. Fused Residual + RMSNorm:** слияние операции сложения остаточных связей (x + residual) и вычисления среднеквадратичной нормализации в единый проход по памяти.
- **4. Оценка на новых GPU и bfloat16:** тестирование решения на архитектурах Ada Lovelace (RTX 4090, L4) и Hopper (H100) с нативным bfloat16.